Fiche Technique d'Architecture — Stack & Composants

Ce document récapitule les technologies et frameworks retenus pour chacun des composants de l'architecture de l'orchestrateur.

1. Frontend Web

- **Rôle :** Interface opérateur (suivi des missions, alertes en temps réel, filtres et console de dialogue avec l'assistant IA).
- **Stack technique :**
- **Framework :** Vite React / Typescript
- **UI & Composants :** Tailwind CSS + shadcn/ui
- **Hébergement :** Azure Static Web Apps
- **Authentification :** Microsoft Entra ID (MSAL / OIDC)
- **Communication :** Fetch REST + WebSocket / SSE pour le flux temps réel

2. Ops API (Passerelle Applicative)

- **Rôle :** Point d'entrée sécurisé pour les opérateurs et le frontend, gestion de l'authentification/autorisations (RBAC) et streaming d'événements.
- **Stack technique :**
- **Langage & Framework :** Python, FastAPI
- **Temps réel :** Starlette SSE / WebSockets
- **Sécurité & Auth :** Azure Entra ID (validation JWT / OAuth2)
- **Déploiement :** Azure Container Apps (ACA - Ingress public)

3. Monitoring Worker (Moteur Déterministe)

- **Rôle :** Polling des missions via MCP, évaluation déterministe des règles YAML (sans IA), cycle de vie des alertes, déduplication, cooldowns et déclenchement des notifications.
- **Stack technique :**
- **Langage :** Python (worker asynchrone / `asyncio`)
- **Moteur de règles :** Moteur interne basé sur Pydantic v2 (validation & parsing des règles YAML)
- **Concurrence & Verrous :** PostgreSQL Advisory Locks (`pg_try_advisory_lock` pour garantir une instance active unique)
- **Déploiement :** Azure Container Apps (ACA - Sans ingress)

4. Agent Service (Enquêteur & Assistant IA)

- **Rôle :** Exécution asynchrone des enquêtes d'alertes complexes et agent conversationnel opérateur.
- **Stack technique :**
- **Orchestration Agentique :** LangGraph / LangChain
- **Client MCP :** SDK MCP Python (`mcp`) ou framework `FastMCP`
- **Validation & Structuration :** Pydantic v2 (décodage contraint / JSON schema pour `submit_assessment`)
- **Déploiement :** Azure Container Apps (ACA - Ingress interne)

5. MCP Server (Passerelle TMS)

- **Rôle :** Unique frontière d'intégration avec le TMS externe ; expose des outils métier sémantiques (tracking, missions, contacts) en protocole standardisé.
- **Stack technique :**
- **Protocole :** Model Context Protocol (MCP) — transport HTTP Streamable (SSE/POST)
- **Framework :** Python (FastAPI / `mcp` server SDK) ou Framework `FastMCP`
- **Client HTTP :** `httpx` (gestion des retries, timeouts, rate limits du TMS)
- **Déploiement :** Azure Container Apps (ACA - Ingress interne)

6. LiteLLM Proxy (Passerelle d'Inférence)

- **Rôle :** Point d'accès unifié aux modèles de langage, gestion du fallback (secours), quotas, circuit breaker et routage des traces d'observabilité.
- **Stack technique :**
- **Moteur :** LiteLLM Proxy (Image Docker officielle)
- **Cache / Quotas partagés :** Redis
- **Déploiement :** Azure Container Apps (ACA - Ingress interne)

7. Inférence LLM

- **Rôle :** Moteur d'intelligence pour le raisonnement, les enquêtes et la synthèse.
- **Stack technique (2 options interchangeables via LiteLLM) :**
- **Option 1 (Auto-hébergé / VNet) :** vLLM sur VM Azure GPU (ex. A10 ou A100) avec modèles type Qwen (ex. Qwen-2.5-14B / Qwen-3.8-27B) avec Prefix Caching activé.
- **Option 2 (Managée sans rétention) :** API Cloud compatible OpenAI (hébergement UE, facturation au token, zero data retention).

8. Observabilité (Langfuse)

- **Rôle :** Traçage complet des appels LLM (latence, coûts, tokens, inputs/outputs, évaluations et feedback).
- **Stack technique :**
- **Composants :** Langfuse Web + Langfuse Worker (Docker / ACA)
- **Base analytique :** ClickHouse (déployé sur VM Azure)
- **Base applicative :** PostgreSQL
- **File d'ingestion :** Redis

9. Données & Stockage (Database & Storage)

- **Rôle :** Persistance opérationnelle, gestion de l'état, caches et stockage d'objets.
- **Stack technique :**
- **Base relationnelle (Cœur métier) :** Azure Database for PostgreSQL Flexible Server (tables d'alertes, verrous, files d'enquêtes `FOR UPDATE SKIP LOCKED`, checkpoints LangGraph)
- **In-Memory & Queues :** Azure Managed Redis (support technique Langfuse & LiteLLM)
- **Analytique traces :** ClickHouse
- **Object Storage :** Azure Blob Storage (rapports, exports, événements bruts Langfuse)

10. Messagerie & Notifications

- **Rôle :** Envoi multi-canal des alertes selon la criticité et les paliers de management (M+0 à M+n).
- **Stack technique :**
- **Canaux :** SMS, Appels voix automatisés (ex. Twilio / Infobip / Azure Communication Services), E-mail (SendGrid / ACS).
- **Pattern d'envoi :** Table Outbox PostgreSQL pour garantir un envoi asynchrone sans bloquer la boucle de surveillance.
